

TP2 — MCP météo, skills, subagents et hooks

Durée estimée : 3h–4h **Niveau :** Débutant–Intermédiaire **Prérequis :** TP1 complété, `pip install mcp httpx` **Format :** Projet fil rouge — vous construisez un assistant météo progressivement **Ce que vous allez produire :** un MCP fonctionnel, 2 skills actives, un hook d’audit, un golden file

Mise en place

Copiez le dossier de support dans votre espace de travail :

```
cp -r tp_claude_debutant_dev/support ~/meteo-agent
cd ~/meteo-agent
mkdir -p .claude/commands tests
cp support/CLAUDE.md ./CLAUDE.md
claude
```

Partie 1 — Connecter le MCP météo

Le MCP (Model Context Protocol) permet à Claude d'appeler des outils externes comme s'ils étaient des outils natifs.

Exercice 1 — Configurer le serveur MCP

Étape 1. Copiez la configuration MCP :

```
mkdir -p .claude
cp support/settings.json .claude/settings.json
```

Étape 2. Vérifiez que le serveur démarre sans erreur :

```
python support/scripts/mcp_meteo.py
```

Vous devez voir le serveur démarrer. Arrêtez-le avec `Ctrl+C`.

Étape 3. Relancez Claude Code et tapez ce prompt :

```
Quels outils MCP as-tu à disposition ? Liste-les avec leur description.
```

Résultat attendu : Claude doit mentionner `mcp__meteo__get_weather` et `mcp__meteo__get_forecast`.

Si les outils n'apparaissent pas : vérifiez que le chemin dans `.claude/settings.json` pointe bien vers `./support/scripts/mcp_meteo.py`.

Exercice 2 — Tester la skill fetching-weather

Étape 1. Activez la skill :

```
cp support/skills/fetching-weather.md .claude/commands/fetching-weather.md
```

Étape 2. Testez avec ce prompt :

```
Quelle est la météo à Paris ?
```

Observez : Claude doit appeler `mcp__meteo__get_weather` et afficher un tableau structuré.

Étape 3. Testez la gestion d'erreur :

```
Quelle est la météo à Xyzabc123 ?
```

Résultat attendu : Claude signale que la ville est inconnue, sans inventer de données.

Étape 4. Testez avec plusieurs villes :

```
Compare la météo à Paris, Berlin et Madrid.
```

Observez : Claude appelle l'outil 3 fois et présente un tableau comparatif.

Exercice 3 — Compléter la skill recommending-activities

Étape 1. Activez la skill :

```
cp support/skills/recommending-activities.md .claude/commands/recommending-activities.md
```

Étape 2. Testez-la :

```
Que faire à Lyon aujourd'hui ?
```

Observez : Claude récupère la météo puis suggère 3 activités avec justification.

Étape 3. Personnalisez la skill en tapant ce prompt :

```
Dans le fichier .claude/commands/recommending-activities.md, ajoute une règle
dans le tableau "Règles de sélection d'activités" pour le cas suivant :
- Neige (description contient "snow" ou "blizzard") → activités : café cosy,
  musée, balade photo dans la neige si vent < 20 km/h.
Garde le format tableau existant.
```

Étape 4. Testez la nouvelle règle :

Que faire s'il neige à Montréal ?

Exercice 4 — Grille de tests de déclenchement

Tapez chacun de ces 10 prompts un par un et notez dans le tableau si la skill se déclenche :

#	Prompt à taper	Skill attendue	Se déclenche ?
1	Météo à Toulouse ?	fetching-weather	
2	Il fait quel temps à Rome ?	fetching-weather	
3	Température actuelle à Oslo	fetching-weather	
4	Que faire à Nice aujourd'hui ?	recommending-activities	
5	Activités par temps de pluie à Nantes ?	recommending-activities	
6	Quelle est la capitale de l'Espagne ?	aucune	
7	Écris un email à mon client	aucune	
8	Prévisions pour la semaine à Lille	aucune (skill non dispo)	
9	Quoi faire à Marseille si il pleut ?	recommending-activities	
10	Temp en degrés à Bordeaux	fetching-weather	

Ensuite, tapez ce prompt pour analyser les éventuels mauvais déclenchements :

Parmi ces 10 prompts, les numéros 6, 7 et 8 ne devaient déclencher aucune skill. Si l'un d'eux en a déclenché une, explique pourquoi et comment corriger la description dans le SKILL.md concerné pour éviter ce faux positif.

Partie 2 — Subagents

Un subagent est une sous-tâche déléguée par Claude à lui-même avec un contexte isolé. Utile pour paralléliser et décomposer des tâches complexes.

Exercice 5 — Requêtes parallèles

Étape 1. Version séquentielle (un seul prompt) :

Donne-moi la météo actuelle des 5 villes suivantes dans un tableau comparatif :
Paris, Londres, Berlin, Madrid, Rome.

Notez le temps approximatif et la structure du résultat.

Étape 2. Version avec pipeline explicite :

Pour récupérer la météo de 5 villes, décompose la tâche ainsi :

- Sous-tâche A : `mcp__meteo__get_weather` pour Paris
- Sous-tâche B : `mcp__meteo__get_weather` pour Londres
- Sous-tâche C : `mcp__meteo__get_weather` pour Berlin
- Sous-tâche D : `mcp__meteo__get_weather` pour Madrid
- Sous-tâche E : `mcp__meteo__get_weather` pour Rome

Exécute les 5 sous-tâches puis synthétise dans un tableau unique :

Ville	Temp (°C)	Conditions	Vent (km/h)
-------	-----------	------------	-------------

Étape 3. Répondez par écrit :

- Quelle version produit un tableau plus structuré ?
- Dans quel cas les subagents explicites sont-ils utiles ? (Indice : volumétrie, isolation du contexte, débogage)

Exercice 6 — Pipeline orchestrateur → recommandation

Étape 1. Tapez ce prompt qui décrit un pipeline en deux étapes :

Analyse météo en deux étapes pour Lyon :

Étape 1 – Données réelles : appelle `mcp__meteo__get_weather` pour Lyon et affiche les résultats bruts (température, conditions, vent, humidité).

Étape 2 – Recommandations : en utilisant exclusivement les données de l'étape 1 (ne pas inventer de conditions météo), génère 3 recommandations d'activités. Chaque recommandation doit citer une valeur chiffrée de l'étape 1 (ex: "avec 18°C et vent à 12 km/h, le vélo est confortable").

Présente les deux étapes séparément dans ta réponse.

Observez : les recommandations de l'étape 2 doivent citer des données concrètes de l'étape 1, pas des formulations génériques.

Étape 2. Comparez avec la version compacte :

Recommande-moi des activités à Bordeaux aujourd'hui.

Notez la différence de précision dans les justifications météo.

Exercice 7 — Gestion d'erreur dans le pipeline

Étape 1. Testez une ville inconnue dans le pipeline :

Je veux des recommandations d'activités pour "Zrtplk".
Étape 1 : essaie de récupérer la météo.
Étape 2 : si la météo est disponible, recommande des activités.
Si la ville est inconnue à l'étape 1, explique pourquoi tu ne peux pas passer à l'étape 2 et ne génère pas de recommandations fictives.

Résultat attendu : Claude bloque à l'étape 1 et n'invente pas de recommandations.

Étape 2. Testez le fallback sans données temps réel :

Imagine que le serveur météo est indisponible en ce moment.
Sans données météo réelles, que peux-tu me proposer comme activités à Paris pour un mois de juin typique ?
Signale clairement dans ta réponse que tu utilises des données historiques approximatives et non des mesures en temps réel.

Observez : Claude doit qualifier explicitement ses réponses ("données historiques", "conditions typiques", etc.).

Étape 3. Améliorez la skill pour gérer ces cas :

Dans `.claude/commands/fetching-weather.md`, ajoute une section `## Gestion d'erreur` avec ces 3 règles :

1. Ville inconnue → afficher "Ville introuvable : [nom]" + proposer 2 orthographes alternatives
2. MCP indisponible → afficher "Données météo temporairement indisponibles" et ne pas continuer vers des recommandations
3. Données aberrantes (temp > 60°C ou < -80°C) → afficher "Données suspectes, vérification manuelle recommandée"

Exercice 8 — Comparaison avec et sans décomposition explicite

Étape 1. Version directe :

Pour Paris, Berlin et Tokyo : donne la météo actuelle et recommande 2 activités par ville. Présente tout dans un tableau.

Étape 2. Version avec pipeline explicite :

Rapport météo + activités pour Paris, Berlin et Tokyo.

Phase 1 – Récupération des données météo :
Appelle `mcp__meteo__get_weather` pour Paris, Berlin et Tokyo.
Affiche les résultats bruts de chaque appel.

Phase 2 – Recommandations basées sur les données réelles :
Pour chaque ville, génère 2 activités. Chaque activité doit citer une valeur chiffrée de la phase 1 (température ou vent).

Phase 3 – Tableau de synthèse :

Ville	Temp	Conditions	Activité 1	Activité 2
-------	------	------------	------------	------------

Étape 3. Répondez par écrit :

- Les recommandations de la version explicite sont-elles plus précises ?
- À partir de combien de villes le pipeline explicite vaut-il l'effort de rédaction ?
- Quel est l'inconvénient de la version explicite pour un utilisateur non-technique ?

Partie 3 — Hooks et permissions

Les hooks sont des commandes shell déclenchées automatiquement par le harness. Ils ne sont pas contrôlés par Claude — ils s'exécutent même si Claude ne le sait pas.

Exercice 9 — Hook d'audit PostToolUse

Étape 1. Vérifiez que le hook est configuré :

```
cat .claude/settings.json | grep -A8 "PostToolUse"
```

Vous devez voir la configuration qui écrit dans `~/.claude/meteo-audit.log` .

Étape 2. Déclenchez un appel MCP :

```
Météo à Amsterdam ?
```

Étape 3. Vérifiez le log :

```
cat ~/.claude/meteo-audit.log
```

Résultat attendu : une ligne JSON avec le timestamp et le nom de l'outil.

Étape 4. Envoyez trois autres requêtes météo puis vérifiez l'accumulation :

```
Météo à Tokyo ?
```

```
Météo à Sydney ?
```

```
wc -l ~/.claude/meteo-audit.log
```

Vous devez voir au moins 3 lignes (une par appel MCP).

Exercice 10 — Hook PreToolUse de validation

Étape 1. Tapez ce prompt pour ajouter un hook de validation :

```
Dans .claude/settings.json, ajoute un hook PreToolUse qui :  
- s'applique uniquement au matcher "mcp_meteo_get_weather"  
- exécute ce script bash exact :  
  if echo "$CLAUDE_TOOL_INPUT" | grep -qE '[0-9]'; then echo "ERREUR : le nom de ville ne doit pas contenir de chiffres" >&2; exit 1; fi  
- place ce hook dans une nouvelle section "PreToolUse" (au même niveau que "PostToolUse")
```

Étape 2. Redémarrez Claude Code, puis testez avec une ville invalide :

```
Météo à Paris75 ?
```

Résultat attendu : le hook bloque l'appel et Claude signale l'erreur.

Étape 3. Vérifiez qu'une ville valide passe :

```
Météo à Paris ?
```

Résultat attendu : fonctionne normalement.

Exercice 11 — Permissions minimales

Étape 1. Testez une commande déjà interdite :

```
Peux-tu supprimer le fichier tests/vide.txt avec rm ?
```

Observez : Claude doit signaler que la commande est interdite ou demander une confirmation.

Étape 2. Ajoutez une interdiction supplémentaire :

```
Dans .claude/settings.json, ajoute "Bash(pip install *)" à la liste  
des commandes interdites dans permissions.deny.  
Je ne veux pas que Claude installe des paquets sans ma validation.
```

Étape 3. Vérifiez la modification :

```
cat .claude/settings.json | grep -A8 "deny"
```

Étape 4. Testez l'interdiction :

```
Installe le paquet requests avec pip.
```

Résultat attendu : Claude signale l'interdiction ou demande une confirmation explicite.

Exercice 12 — Golden file de non-régression

Contexte : Un golden file capture la structure attendue d'une réponse. Il permet de détecter si une mise à jour du modèle ou de la skill change le comportement.

Étape 1. Créez le golden file :

Exécute la skill `fetching-weather` pour Paris et crée le fichier `tests/golden-paris.txt` avec ce contenu exact (remplace [VALEURS] par les vraies données récupérées) :

```
---
prompt: "Quelle est la météo à Paris ?"
sections_attendues:
  - "Météo actuelle"
  - "Température"
  - "Conditions"
  - "Humidité"
  - "Vent"
  - "wttr.in"
exemple_sortie_reelle: |
  [colle ici la vraie sortie de la skill]
structure_ok: true
---
```

Étape 2. Créez le script de test de régression :

Crée le fichier `tests/test_regression.sh` qui :

1. Lance le prompt "Quelle est la météo à Paris ?" via : `claude -p "Quelle est la météo à Paris ?"`
2. Vérifie que la sortie contient les 6 mots-clés du golden file :
"Météo actuelle", "Température", "Conditions", "Humidité", "Vent", "wttr.in"
3. Pour chaque mot-clé manquant, affiche "MANQUANT : [mot-clé]"
4. Affiche "✅ OK – structure conforme" si tout est présent
5. Affiche "❌ RÉGRESSION DÉTECTÉE" si au moins un mot-clé manque
6. Retourne le code de sortie 0 si OK, 1 si régression

Étape 3. Rendez le script exécutable et lancez-le :

```
chmod +x tests/test_regression.sh
./tests/test_regression.sh
```

Résultat attendu : ✅ OK – structure conforme

Étape 4. Simulez une régression :

Dans `.claude/commands/fetching-weather.md`, supprime la section "## Format de sortie" et remplace-la par : "Réponds librement selon le contexte."

Relancez le test :

```
./tests/test_regression.sh
```

Résultat attendu : ❌ RÉGRESSION DÉTECTÉE avec les sections manquantes listées.

Restaurez la skill :

Restaure la section "## Format de sortie" dans `.claude/commands/fetching-weather.md` avec le tableau original (Indicateur / Valeur avec les 5 lignes : Température, Conditions, Humidité, Vent, Visibilité).

Livable

À la fin du TP2, votre dossier `~/meteo-agent` doit contenir :

```
~/meteo-agent/
├── CLAUDE.md
├── .claude/
│   ├── settings.json      ← MCP + hooks PostToolUse + PreToolUse + permissions
│   └── commands/
│       ├── fetching-weather.md      ← skill + section gestion d'erreur
│       └── recommending-activities.md ← skill + règle neige ajoutée
├── support/
│   ├── scripts/mcp_meteo.py
│   └── skills/
├── tests/
│   ├── golden-paris.txt      ← sortie de référence
│   └── test_regression.sh    ← script de non-régression
```

Et `~/.claude/meteo-audit.log` dans votre home avec les lignes d'audit accumulées.